

agent-memtrace: 階段感知的 coding agent sandbox 記憶體特徵化

資料產生 2026-10-06 03:03:18 · git c39bea0 · 預先登記 sha256 a4c949b9cea2d25b...

草稿: A 的監測窗預定到 2026-10-08 09:00, 本版只含 2026-10-06 01:18 至 2026-10-06 03:03 的資料; A 的數字與判定在最終版會改變。B 已完成。

摘要 以無需 root 的 /proc 取樣與 clear_refs, 量測 coding agent 行程樹在等模型、執行工具、等使用者三階段的記憶體。在真實 Claude Code session 與 30 次 SWE-bench Lite 執行中, ≥ 5 秒等模型區間結束時未被觸碰的記憶體比例中位數為 74% (整棵行程樹) 與 90% (SWE-bench sandbox); 等模型區間長度中位數為 7.2 與 2.1 秒, SWE-bench 有 97% 的記憶體尖峰發生在工具執行期間。冷記憶體比例高, 但過半的等模型區間短於 10 秒。

1. 方法

1.1. 量測器

量測器 (memtrace/sampler.py, 僅用標準函式庫) 以一般使用者身分執行, 不需 root、不改 kernel、不裝系統套件。每個行程讀 /proc/<pid>/smaps_rollup (Rss、Pss、Referenced、Swap)、/proc/<pid>/io (rchar、wchar、read_bytes、write_bytes)、/proc/<pid>/stat (utime+stime+cutime+cstime、page fault)。已回收子行程的 I/O 與 CPU 會併入父行程的計數器 (實測確認), 故對樹內存活成員加總即得累計值。B 的 docker exec 行程由樹外的 containerd-shim 回收, 因此 sandbox 的 I/O 與 CPU 改用容器 cgroup 的 io.stat、cpu.stat, 並另記 memory.current、memory.stat。

冷記憶體: 每次即時偵測到階段轉換, 就對樹內所有成員寫 1 到 /proc/<pid>/clear_refs, 清除 accessed 位元; 之後任一樣本的 Referenced 即為「自本階段開始以來被觸碰過的頁」, 冷 = Rss - Referenced。區間結束前最後一個 1 Hz 樣本給出該區間的冷比例。檔案快取側每 10 秒以 vmtouch (mmap + mincore, 不觸碰頁面) 估工作目錄的常駐頁數; 單次掃描超過 0.5 秒的目錄改為每 20 倍掃描時間掃一次, 以限制 CPU 開銷。

1.2. 階段切割與時間對齊

memtrace/phases.py 的狀態機同時用於即時觸發 clear_refs 與事後切割區間, 兩者邊界一致。A 讀 Claude Code session JSONL (另支援 Codex rollout), 只取時間戳與記錄型別, 不存任何訊息內容。B 由插樁的 mini-SWE-agent 迴圈在每次模型呼叫與工具執行前後寫事件。量測器樣本、JSONL 時間戳與 B 事件都是同一主機的 time.time(), 故區間邊界與樣本的對齊誤差即時間戳本身的誤差; 即時偵測延遲只影響 clear 的時機 (A: 中位數 0.25 秒, p95 1.05 秒, 93% 小於 1 秒), 並以每個區間的「覆蓋率」(clear 後首個可用樣本到區間結束占區間長度的比例) 報告。

1.3. 工作負載

- **A (真實 session):** systemd 使用者服務在 ws4 上監測本使用者所有 Claude Code / Codex 行程樹, 共 3 棵樹、3 個 session、合計 5.2 樹·小時。
- **B (SWE-bench Lite):** mini-SWE-agent 2.4.6 在 rootless Docker 29.8.2 的官方 SWE-bench 映像中執行, 共 30 次執行、24 題; 模型 ("deepseek/deepseek-flash",), 總花費 1.75 USD。映像在量測窗外預先拉取。

1.4. 預先登記 (2026-10-06 凍結, 於 A、B 資料收集前提交)

1.4.1. 假設 (原文, 門檻不改; 不成立照實報告)

- **H1:** 在長度 ≥ 5 秒的等模型區間內, 行程樹 RSS 中未被觸碰 (Referenced=0) 的比例中位數 $\geq 50\%$ 。
- **H2:** 等模型區間長度中位數 ≥ 10 秒。
- **H3:** 每個任務的記憶體尖峰落在工具執行區間內的比例 $\geq 80\%$ (重現 AgentCgroup 的「尖峰由工具呼叫驅動」)。

1.4.2. 行程樹

- **A:** 本使用者在 ws4 上每個 Claude Code (comm claude) 或 Codex (comm codex*) 行程為根, 加上以 /proc/<pid>/task/*/children 遞迴收集的全部子孫, 每次取樣重新收集。位於另一個已追蹤根之下的 agent 行程併入外層樹。量測器自身及其子行程排除。
- **B:** 主要對象為 sandbox, 即 SWE-bench 容器 cgroup 的 cgroup.procs 內全部行程 (docker exec 產生的行程不是容器 init 的子孫, 故以 cgroup 定義成員, 與 AgentCgroup 相同)。harness (mini-SWE-agent 行程及其子孫, 含 docker CLI) 另列為次要分析, 不進入 H1-H3 主判定。

1.4.3. 取樣與量

- 1 Hz, 落在整數秒 (time.time()), 稱 primary 樣本。即時偵測到 tool_exec 期間另以 10 Hz 補樣 (primary=0)。H1-H3 主判定只用 primary 樣本; 含補樣的結果僅作敏感度分析。

- 每行程讀 `smaps_rollover` (`Rss`、`Pss`、`Referenced`、`Swap` 等) 、`io`、`stat`。樹的 `Rss`、`Referenced` 為成員加總 (kB)。冷比例 = $\Sigma(Rss - Referenced) / \Sigma Rss$ 。

1.4.4. 階段

- **A:** 由 `session.JSONL` 記錄時間戳以 `memtrace/phases.py` 狀態機切割 (即時觸發與事後切割用同一份程式碼)。`tool_exec` = [回應中的 `tool_use` 記錄 → 該批最後一個 `tool_result`]; `model_wait` = [`user` 提示或最後一個 `tool_result` → 下一筆含 `tool_use` 或結束該輪的 `assistant` 記錄]; `user_wait` = [該輪最後一個 `assistant` 區塊 → 下一個 `user` 提示]。`subagent` (`sidechain`) 屬於父 `session` 的 `tool_exec`。未被模型回應 (或使用者中斷) 關閉的 `model_wait` 標為 `unclosed` 並排除。
- **B:** 由插樁的 `agent` 迴圈事件切割 (同一時鐘): `model_wait` = [`model_start`, `model_end`], `tool_exec` = [`tool_start`, `tool_end`]。B 沒有 `user_wait`。

1.4.5. clear_refs

- 每次即時偵測到階段轉換 (`model_wait`、`tool_exec`、`user_wait` 開始), 對樹內所有成員寫入 1 到 `/proc/<pid>/clear_refs`。一個區間的「主控 clear」為該區間起點轉換所觸發的 clear。

1.4.6. 統計量與判定

- **H1:** 每個長度 ≥ 5 秒、有主控 clear 的 `model_wait` 區間, 取 clear 完成之後、區間結束之前的最後一個 `primary` 樣本計算冷比例; 沒有這種樣本的區間排除並計數。A、B 分別判定: 中位數 ≥ 0.50 為成立。
- **H2:** 所有已關閉、且完整落在監測窗內 (A: 區間起點晚於該樹被掛上的時間) 的 `model_wait` 區間長度中位數 ≥ 10 秒為成立。A、B 分別判定。
- **H3:** 任務定義 — A: 一個使用者回合 (由提示觸發的 `model_wait` 起到下一個 `user_wait` 開始), 且含 ≥ 1 個 `tool_exec` 區間與 ≥ 5 個 `primary` 樣本; B: 一次執行 (原始與重跑各算一次)。尖峰 = 任務內樹 `Rss` 最大的 `primary` 樣本 (同值取最早); 樣本時間落在 `tool_exec` 區間內即算「在工具內」。比例 ≥ 0.80 為成立。A、B 分別判定。
- 次要 (不影響判定): B 的 `harness+sandbox` 合計、B 的 `cgroup memory.current`、含 10 Hz 補樣的版本、`Pss` 版本。

1.4.7. 工作負載與排除

- **A:** A `daemon` 啟動起至 2026-10-08 09:00 (本地時間), 只限 `ws4`; 含建置本工具的這個 `session`。`daemon` 啟動前的試跑資料排除。
- **B:** SWE-bench Lite test split, `id` 排序後以 `random.Random(20261006)` 洗牌, 依序取映像檔拉取成功的前 24 題; 其中前 3 題各重跑 2 次 (共 30 次執行)。同時 3 個執行。`mini-SWE-agent 2.4.6`、內建 `swebench.yaml` 設定, `step limit 100`、`cost limit 1 USD`、`wall-time limit 2400 秒`, 模型為 `DeepSeek flash` 等級 (確切 ID 記於 `raw`)。`rootless Docker 29.8.2` (非 `venv`)。試跑 (`--limit-runs`) 寫入獨立目錄並排除。
- 128 GB 主機容量換算為探索性分析, 不屬預先登記。

1.5. 與原計畫的偏離

- 報告改用 `Typst` (使用者要求); 數字全部由 `Typst` 直接讀 `numbers.json` 排入, 無手抄數字。
- B 使用 `rootless Docker` (計畫中「Docker 可用則用容器」的分支; Docker 依 CSIE 官方流程於本次安裝)。
- 新增 10 Hz 工具期間補樣與 `cgroup` 指標, 僅作次要/敏感度分析 (於凍結前登記)。
- `vmtouch` 週期改為自適應 (試跑顯示固定 10 秒在 26 GB、3.5 萬檔的目錄上耗用約 30% 單核)。

2. 結果

假設	門檻	A 值 (n)	A	B 值 (n)	B
H1 冷比例中位數	≥ 0.50	0.74 (50)	成立	0.9 (233)	成立
H2 等模型長度中位數	≥ 10 s	7.2 s (75)	不成立	2.1 s (1531)	不成立
H3 尖峰在工具內比例	$\geq 80\%$	50% (4)	不成立	97% (30)	成立

預先登記假設的判定。A = 整棵行程樹; B = `sandbox` (容器 `cgroup`)。n 為區間數 (H1、H2) 或任務數 (H3)。

負載	階段	n	長度 s	Rss MiB	Pss MiB	冷比例 (n)
A	<code>model_wait</code>	75	7.2 / 64.2	881 / 916	553 / 579	0.78 / 0.84 (75)
A	<code>tool_exec</code>	70	0.7 / 33.5	897 / 1204	561 / 859	0.73 / 0.97 (22)
A	<code>user_wait</code>	5	1288.8 / 1420.2	825 / 880	501 / 546	0.68 / 0.69 (5)
B sandbox	<code>model_wait</code>	1531	2.1 / 8.3	8 / 57	2 / 41	0.99 / 1 (1515)
B sandbox	<code>tool_exec</code>	1924	0.4 / 20.3	40 / 300	31 / 262	0.07 / 0.99 (989)
B 全部	<code>model_wait</code>	1531	2.1 / 8.3	208 / 257	180 / 222	0.95 / 0.98 (1515)
B 全部	<code>tool_exec</code>	1924	0.4 / 20.3	284 / 541	242 / 477	0.8 / 0.95 (990)

表 1 各階段的中位數 / p95。`Rss`、`Pss` 為區間內 1 Hz 樣本的平均; 冷比例取區間結束前最後一個樣本, n 為有效區間數 (短於 1 秒的區間多半沒有樣本)。B 全部 = `harness + sandbox`。

負載	供給依據	每 sandbox GiB	分層後 GiB	原台數	分層後	增幅
A 整樹	mean	0.84	0.81	136	141	+3.8%
A 整樹	p95	0.9	0.9	127	127	+0%
B harness+sandbox	mean	0.49	0.45	236	258	+9.1%
B harness+sandbox	p95	0.65	0.61	176	187	+6.1%

表 2 探索性換算：128 GB 主機（保留 10%）可容納的 sandbox 數。假設：各 sandbox 獨立、以每 sandbox 的時間平均或 p95 記憶體配置、只在 ≥ 5 秒的等模型區間把「整個區間都沒被觸碰」的頁（區間結束時的 Rss – Referenced）在區間開始時移出且無成本、慢速層容量不限、不計頁快取與跨 sandbox 共享頁。

2.1. 等模型期間的冷記憶體（H1）

A 的整棵行程樹在長度 ≥ 5 秒的等模型區間結束時，冷比例中位數為 0.74（ $n = 50$, p95 0.82），H1 在 A 成立。B 的 sandbox 中位數為 0.9（ $n = 233$ ），H1 在 B 成立；但這些區間結束時 sandbox 的 Rss 中位數只有 8.4 MiB（p95 87.5 MiB）——沒有工具在跑時，sandbox 只剩容器 init 與 agent 留在背景的行程，比例高但絕對量小。容器外的 harness 在同一時刻約 201 MiB，冷比例中位數 0.95；harness + sandbox 合計 0.95。clear 之後到區間結束的覆蓋率中位數：A 96%、B 94%。冷比例與區間長度的關係見圖 2。

2.2. 等模型區間長度（H2）與預取窗口

等模型區間長度中位數：A 7.2 秒（ $n = 75$, p95 64.2 秒），B 2.1 秒（ $n = 1531$, p95 8.3 秒）。H2 在 A 不成立、在 B 不成立。B 使用回應快的 flash 等級模型，每次呼叫通常只有數秒；可用於分層與預取的窗口因此主要落在少數長區間的尾部，而不是典型區間。

2.3. 尖峰位置（H3）與大小

B 有 97% 的執行（ $n = 30$ ）其 sandbox 尖峰落在工具區間內，H3 在 B 成立；加入 10 Hz 補樣為 100%，以 cgroup memory.current 計為 97%。A 只有 4 個符合條件的使用者回合，比例 50%（補樣版本 75%），H3 在 A 不成立。

B 每次執行的 sandbox 尖峰（cgroup memory.current）中位數 0.48 GiB、p95 5.6 GiB、最大 26.8 GiB。9 次執行在 sandbox 內同時有超過 100 個行程（最多 154 個）：Django 的測試執行器預設依可見 CPU 數開 worker，而容器沒有 CPU 限制、看得到主機的 144 核；這類尖峰的大小因此取決於主機核心數，不能直接外推到有 CPU 配額的 sandbox。fork 出的 worker 共享頁面， ΣRss 因而重複計算（尖峰中位數 ΣRss 0.44 GiB，p95 11.1 GiB，對照 Pss p95 5.5 GiB），所以圖 3 改用 Pss 畫尖峰／平均比；H3 判定仍依預先登記用 Rss；改用 Pss 時比例為 B 97%、A 50%。

2.4. 各階段特徵、檔案快取與 I/O

工具區間很短：中位數 A 0.69 秒、B 0.35 秒（表 1）。工作目錄的頁幾乎全部常駐於頁快取：B 的 /testbed（中位數 134 MiB）常駐比例中位數 100%。因此每次工具呼叫的儲存端讀取多半為零：B 的 sandbox cgroup 有 99% 的工具呼叫沒有任何 block 讀取，A 為 81%；A 的 syscall 層讀取中位數則為 1499 KiB（圖 4）。

題目	次	長度 s	尖峰 GiB	等模型中位數 s	冷比例中位數
django_django-13158	0	321	1.05	1.9	0.71
django_django-13158	1	765	26.81	2.7	0.86
django_django-13158	2	511	0.43	1.7	0.99
psf_requests-2317	0	261	0.48	2.4	0.95
psf_requests-2317	1	310	0.07	1.6	0.96
psf_requests-2317	2	232	0.06	1.7	1
sympy_sympy-14308	0	631	0.49	2.3	0.74
sympy_sympy-14308	1	716	0.32	3.2	0.74
sympy_sympy-14308	2	640	0.21	2	1

表 3 B 重跑變異：同一題的三次執行。尖峰為 sandbox cgroup memory.current；冷比例為 ≥ 5 秒等模型區間的中位數。

2.5. 128 GB 主機換算（探索性）

在表 2 的假設下，把 ≥ 5 秒等模型區間中整段未被觸碰的頁移到慢速層，以時間平均配置計，A 每台可多放 3.8%、B（harness + sandbox）多放 9.1%；以 p95 配置計分別為 0% 與 6.1%。增幅小的主因是可分層的時間少：符合條件（ ≥ 5 秒且有有效樣本）的等模型區間只占 A 監測時間的 5% 與 B 執行時間的 17%；A 的大部分時間是等使用者，依計畫不計入（若一併計入等使用者區間，A 的時間平均增幅為 24.5%）。

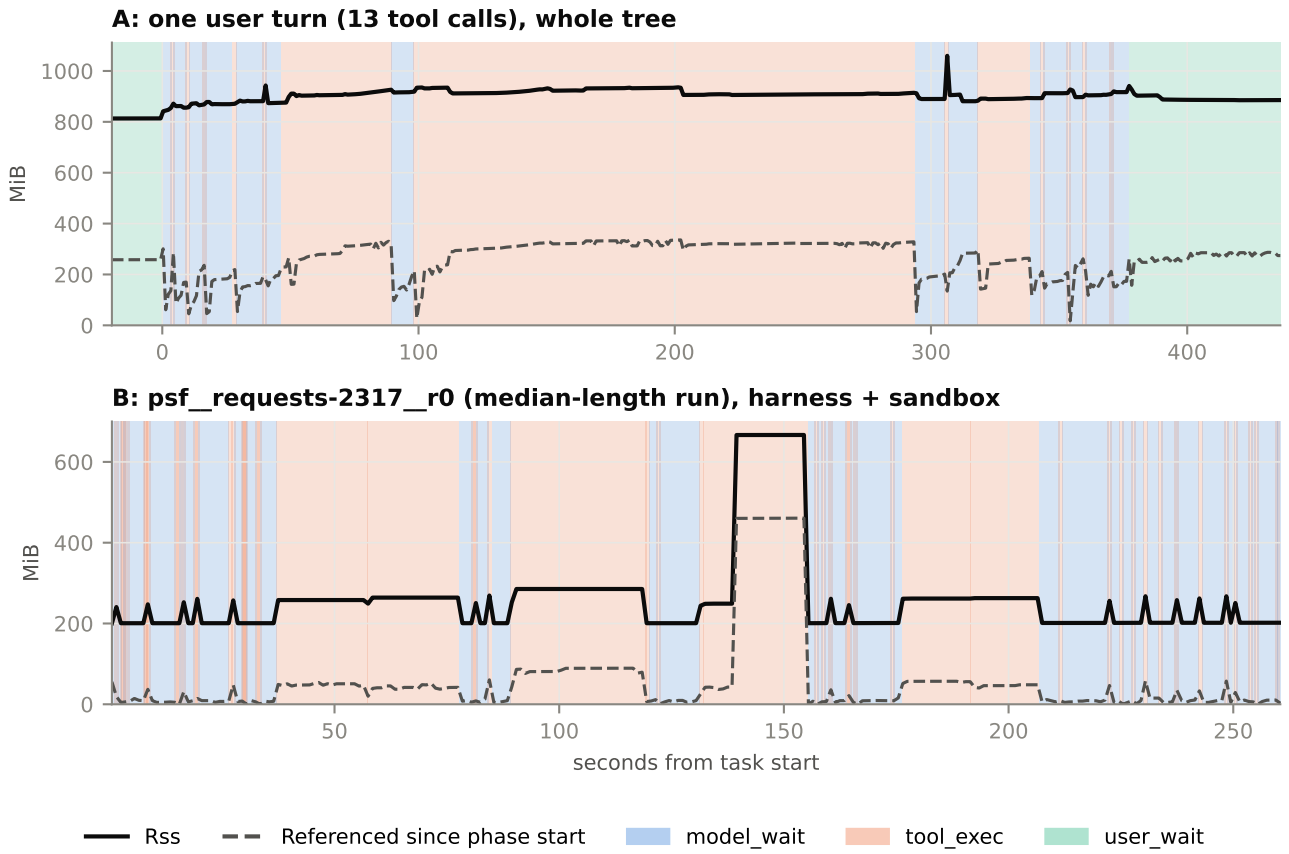


圖 1 單一任務時間線。實線為整樹 (A) 或 harness + sandbox (B) 的 Rss，虛線為自本階段開始以來被觸碰過的頁 (Referenced)；底色為階段。短於圖寬 0.4% 的工具區間為了可見而加寬。

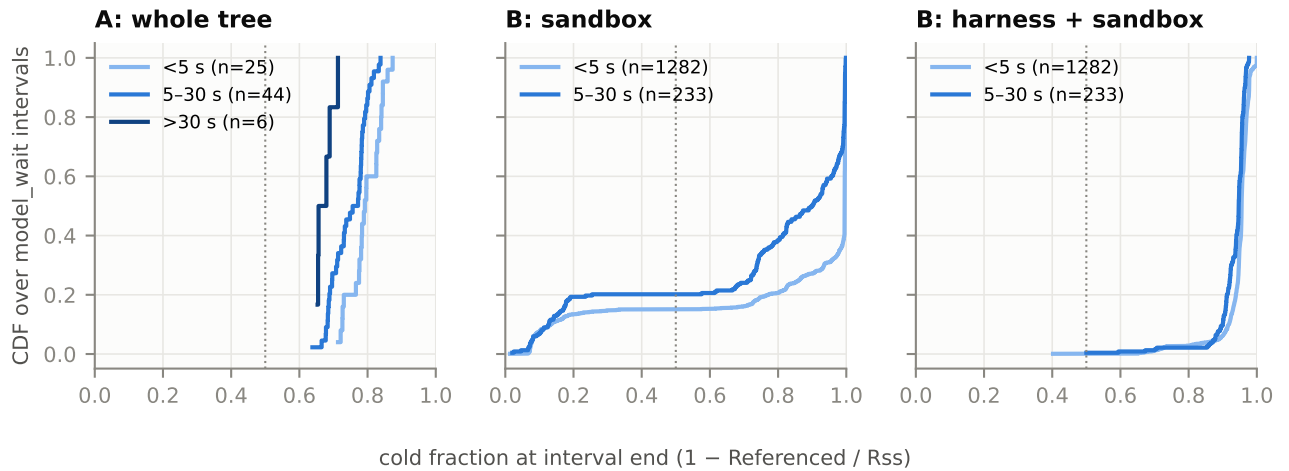


圖 2 等模型區間結束時的冷比例 CDF，依區間長度分組；點線為 H1 門檻 0.5。

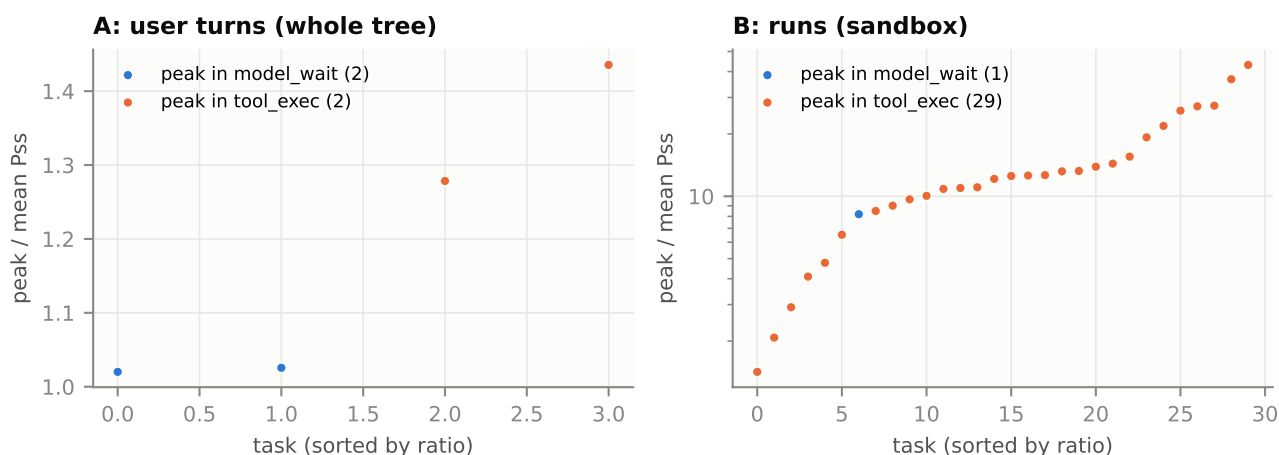


圖 3 每個任務的尖峰／平均 Pss 比，顏色為尖峰樣本所在階段（1 Hz primary 樣本）。

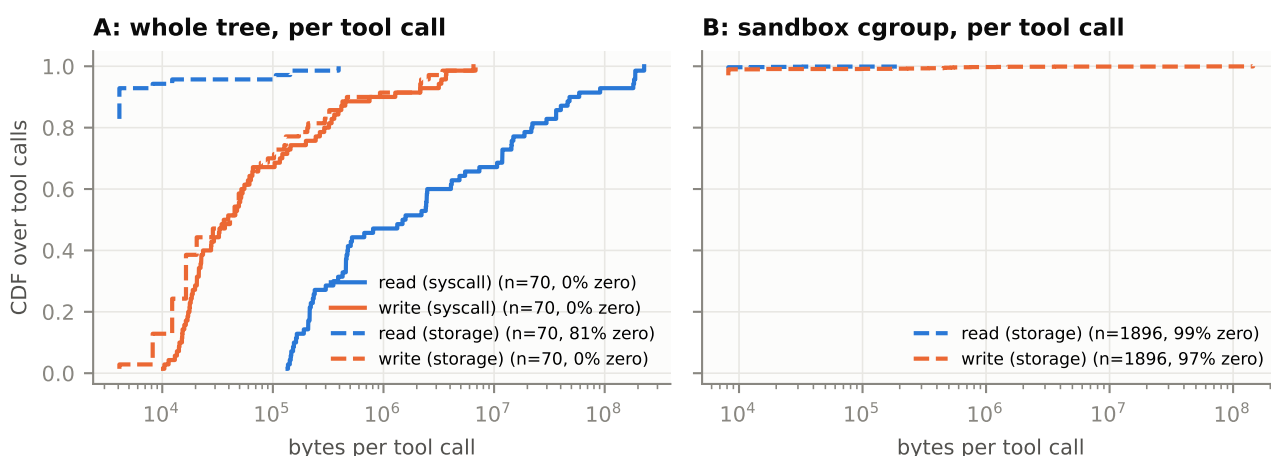


圖 4 每次工具呼叫前後（以最近的樣本夾住）的讀寫位元組 CDF；0 位元組的比例標在圖例中。syscall = rchar/wchar, storage = read_bytes/write_bytes (A) 或 cgroup io.stat (B)。

3. 量測開銷

A 的量測器平均使用 9.7 % 單核 CPU (vmtouch 另 6.3 %)，每次取樣中位數 72 ms (p95 113.7 ms)，自身 RSS 約 22.8 MiB。微基準（目標行程反覆掃 256 MiB，每條件 5 次 × 20 秒）：每秒讀一次 smaps_rollup 使掃頁吞吐量變化 0 %，再加上每秒 clear_refs 為 0.7 %（未量測組的標準差為其平均的 4.2 %）；minor fault 數三組相當（2542、3732、2516）。在 x86 上清 accessed 位元不會使頁面失效，所以 clear_refs 主要成本是一次頁表走訪與 TLB flush，而非額外 page fault；計畫中預期的「少量額外 page fault」在本機未觀察到。單次 smaps_rollup 讀取中位數 1.97 ms、clear_refs 1.14 ms（1 GiB 行程）。

4. 限制

- 取樣粒度：工具區間中位數不到 1 秒（表 1），1 Hz 樣本會漏掉多數短工具的尖峰，使 H3 偏低；10 Hz 補樣依賴即時偵測，而 Claude Code 偶爾延遲寫入 tool_use 記錄，補樣可能晚開始。敏感度結果見結果節。
- Referenced** 的語意：THP 設為 always，一個 2 MiB 大頁只要有一個子頁被碰就整頁算 Referenced，冷比例因此偏保守（低估）。共享頁（函式庫、node 執行檔）在每個行程的 Rss 中各算一次，ΣRss 高於實際占用；Pss 欄位供對照。
- clear** 時機：clear 發生在偵測到轉換之後，區間開頭一小段的觸碰不會被算進 Referenced，冷比例因此略為高估；clear 後到區間結束的覆蓋率中位數為 A 96 %、B 94 %。
- 頁快取：clear_refs 只涵蓋已映射的頁，未映射的頁快取無法以無 root 方式量測是否被觸碰 (idle page tracking 需 root)；B 的容器 cgroup 只被計入它首次讀入的檔案頁，映像層的頁快取多半記在 Docker daemon 名下。mincore 只給常駐量，不給觸碰與否。
- 代表性：A 只含單一使用者、單一主機、約兩天、且包含建置本工具的 session；B 的 harness 在容器外，與 harness 在 sandbox 內的部署（如 AgentCgroup）不同，因此 B 另報 harness + sandbox。B 為單一模型、單一 agent 框架。
- 時鐘：所有時間戳來自同一主機；Claude Code 的 assistant 記錄時間戳是區塊完成時間，user_wait 起點以同一回應最後一個區塊為準。

本報告不推論「agent 很省記憶體」或任何超出上述量測範圍的結論。